

CS & IT ENGINEERING

Programming in C

Functions and Storage Classes
Lec- 03



By- Pankaj Sharma sir



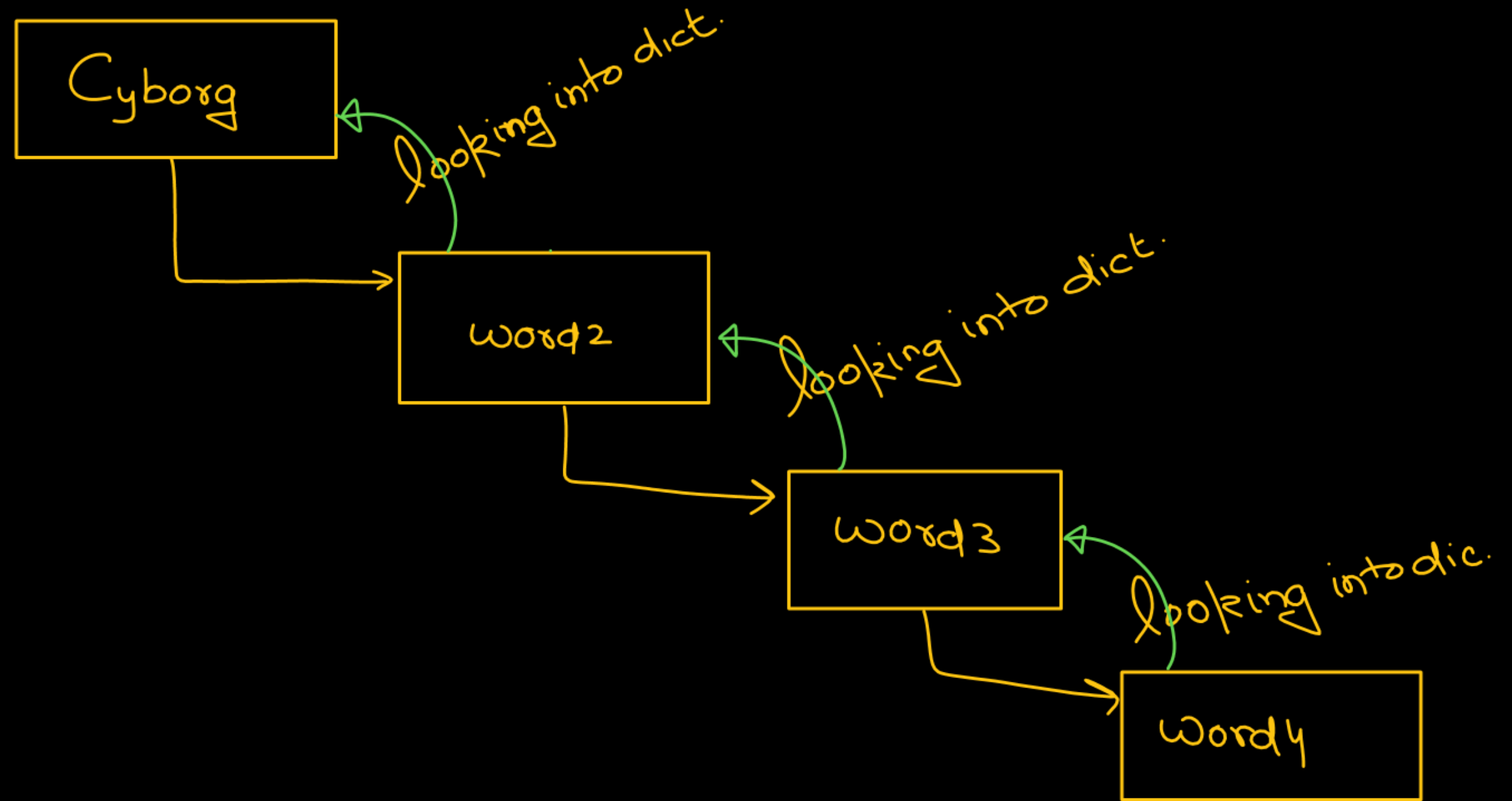
TOPICS TO BE
COVERED



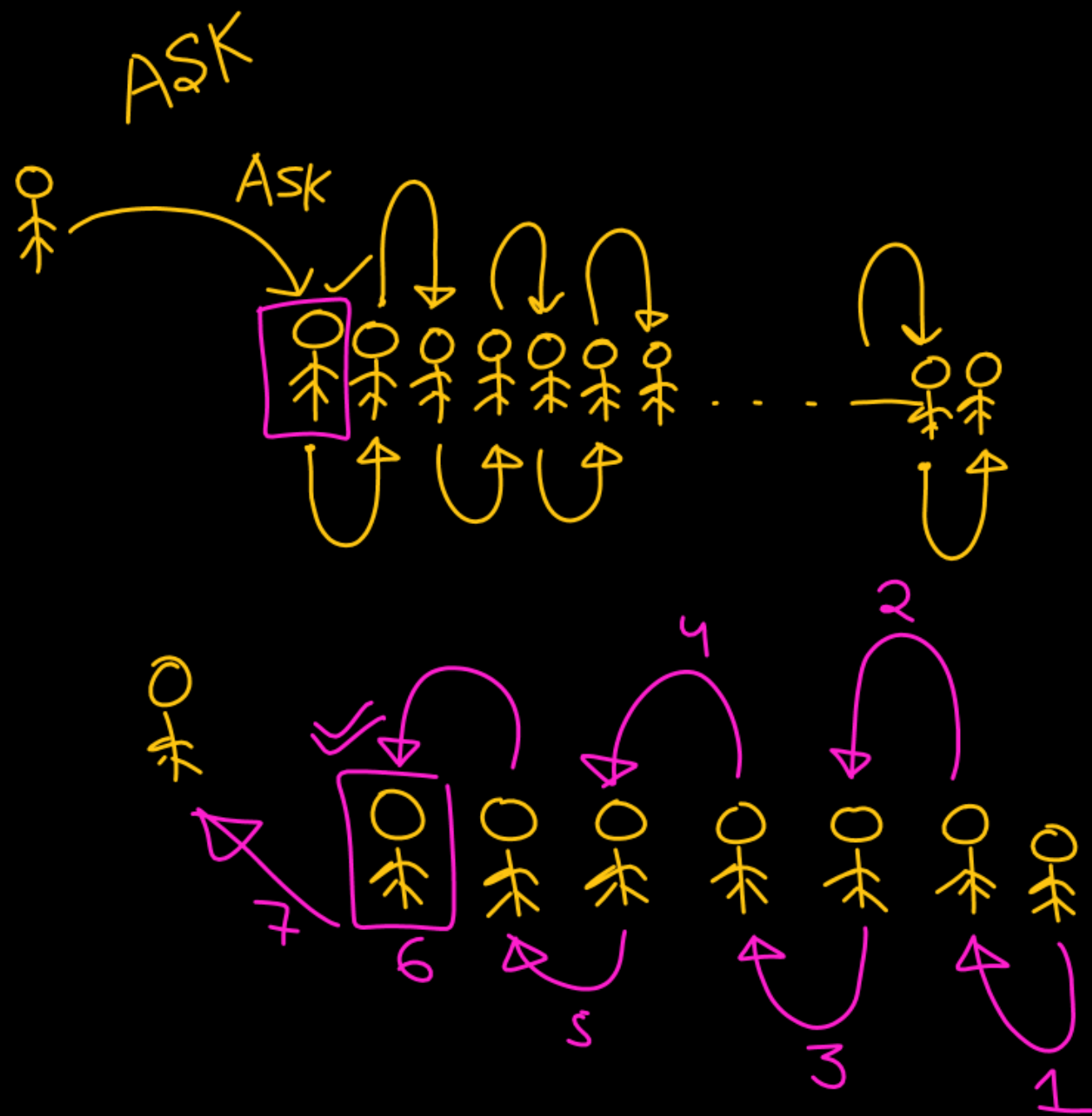
Storage Classes and Recursion

Recursion

Ex 1:



Ex2.



atleast

if (n is small)

{

Easy case

Can be answered directly

No recursion is needed

}

else {

Hard case

i/p is large

Can ~~not~~ be answered directly

Recursion is needed

}

Ex 1

$n > 0$

i/p : $n = 2$

o/p : PankajPankaj

i/p : $n = 3$

o/p : PankajPankajPankaj

i/p : $n = 6$

o/p : PankajPankajPankajPankajPankajPankaj

```
void fun(int n)
{
```

```
    if (n is small)
    {
```

Easy case

No recursion is needed

Can be answered directly

```
    else {
```

n is large

Hard case

Can not be answered directly

Recursion is needed.

```
    }
```

```
}
```

①

$n = 1000$

```
void fun(int n)
{
```

```
    if (  $n == 1$  )
    {
```

```
        printf("Pankaj");
        return;
```

```
    else {
```

n is large

Hard case

Can not be answered directly
Recursion is needed.

```
    }
```

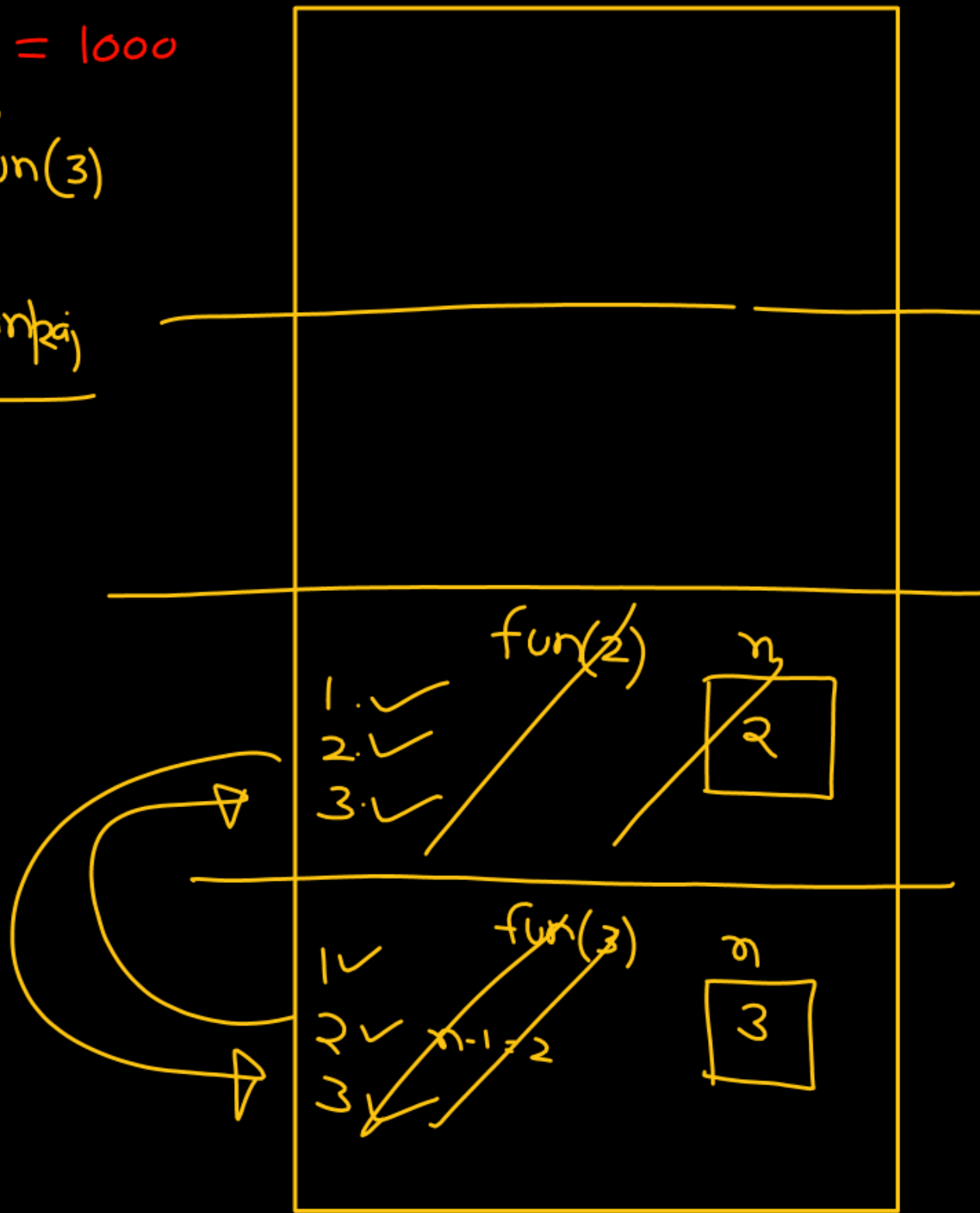
```
}
```

Every rec. call perform
some unit of work and
the remaining is left to
recursion.

$n = 1000$

① $\text{fun}(3)$

Pankaj Pankaj Pankaj



```
void fun(int n)
{
```

```
    if(  $n == 1$  )
```

```
    {
```

```
        printf("Pankaj");
        return;
```

```
    else {
```

```
        1. printf("Pankaj");
```

```
        2. fun(n-1);
```

```
        3. }
```

```
    }
```


base value/base parameters: The values for which the function does not call itself.

$a, n > 0$

a^n

$3^{10} \Rightarrow 3 \times 3 \times 3 \times 3 \times 3 \times 3 \times 3 \times \dots$ 10 times

```
int Power(int a, int n){
```

```
    if( n ? )
```

```
    {
```

Can be answered directly

No recursion is needed

Easy case

```
    }
```

```
else {
```

Recursion

```
    }
```

$a, n > 0$

a^n

$3^{10} \Rightarrow 3 \times 3 \times 3 \times 3 \times 3 \times 3 \times 3 \times \dots$ 10 times

```
int Power (int a, int n) {
```

```
    if (n == 1)
```

```
        return a;
```

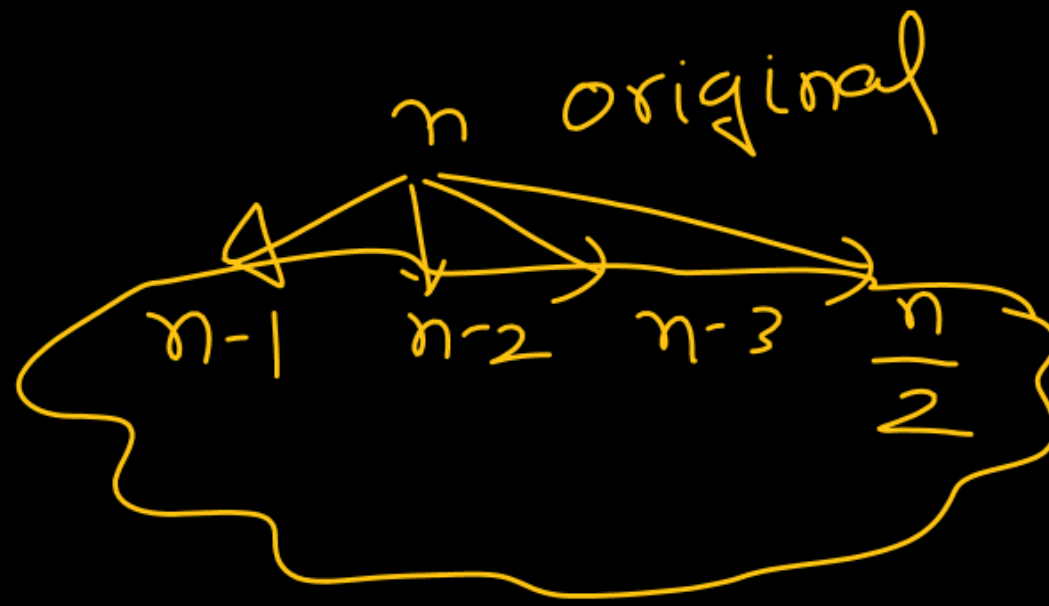
```
    else {
```

```
        return
```

```
        a  $\times a^{n-1}$  Power(a, n-1);
```

```
    }
```

$$\underbrace{3^{100}}_{\uparrow} = 3 \times 3^{99}$$



n > 0

i/p : 1289

o/p : 20

i/p : 925

o/p : 16

i/p : 9

o/p : 9

i/p : 1

o/p : 1

i/p : 12376

o/p : 19

```
int sum(int n)
```

```
{
```

```
if(n ?)
```

```
{
```

⇒ what is small value of n for which we can answer directly

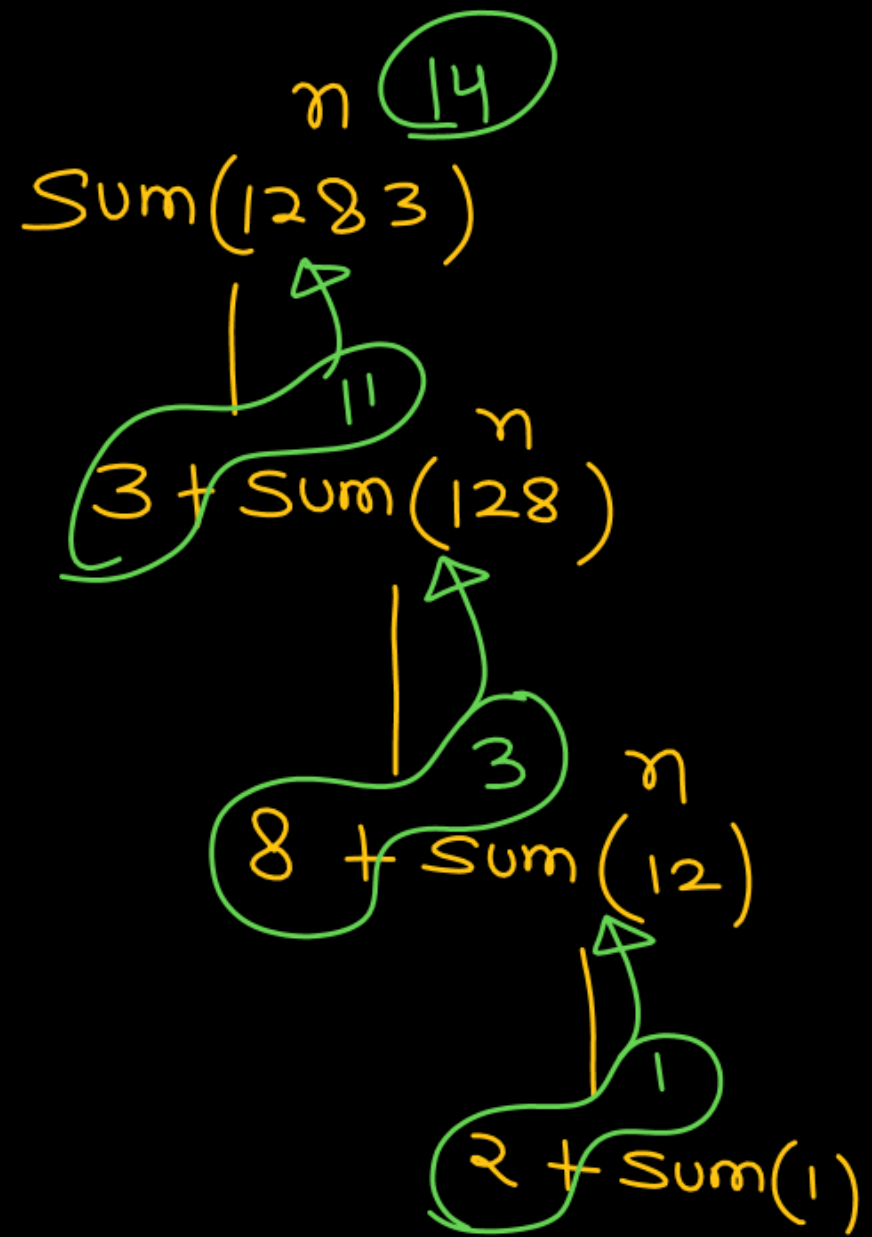
```
}
```

```
else{
```

```
}
```

```
}
```

any
single digit
no. ⇒ same



```
int sum(int n)
{
    if ( n <= 9 )
        return n;

    else {
        return n%10 + sum( n/10 )
    }
}
```

1.

```
void f(int n){
```

```
    if(n>0)
    {
```

```
        1. f(n-1);
```

```
        2. printf("%d", n);
```

```
        3. }
```

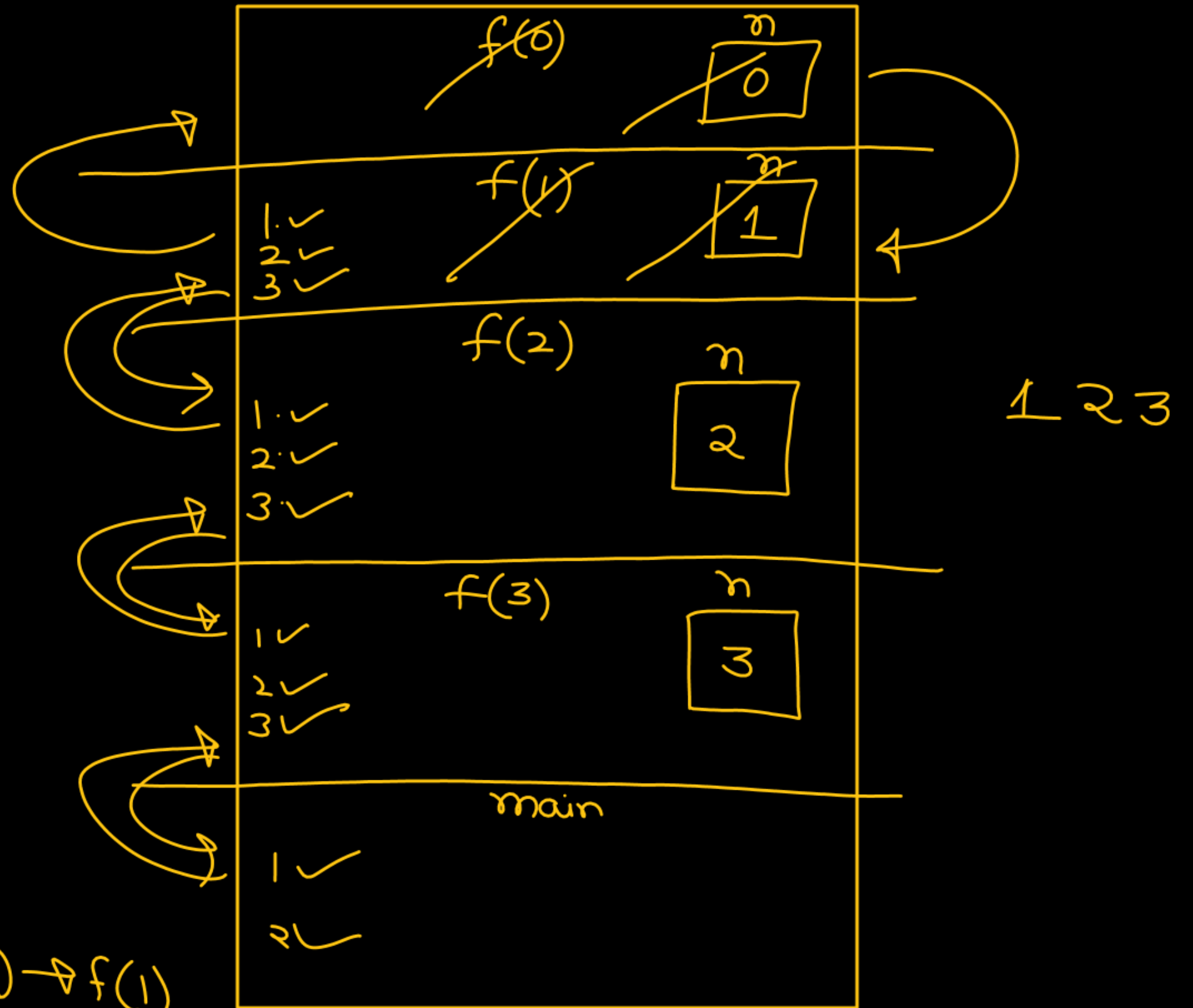
```
    }
```

```
void main(){
```

```
    1. f(3);
```

```
    2. }
```

main() → f(3) → f(2) → f(1)



1.

```
void f(int n){
```

```
    if(n>0)
    {
```

```
        1. f(n-1);
```

```
        2. printf("%d", n);
```

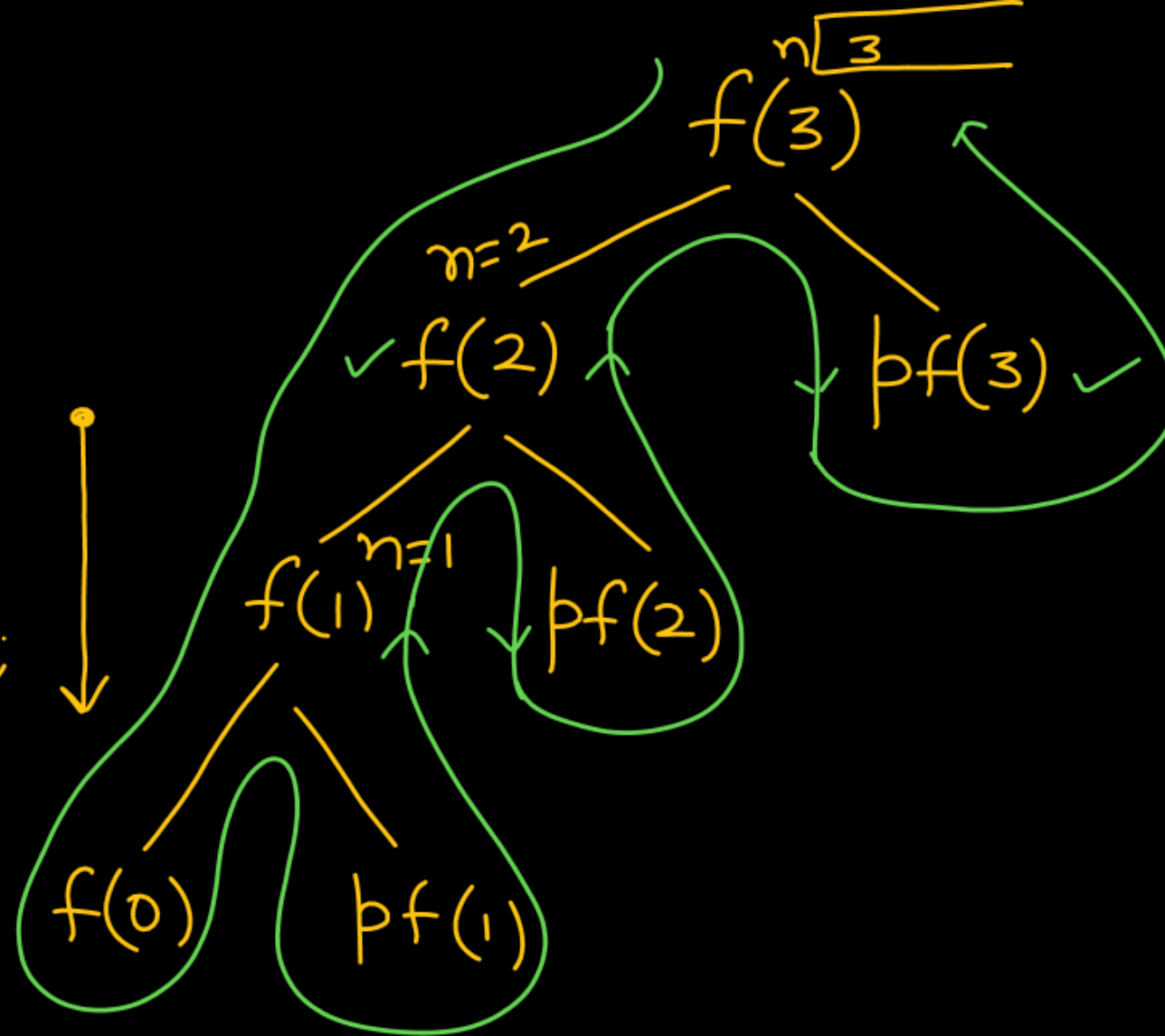
```
        3. }
```

```
    }
```

```
void main(){
```

```
    1. f(3);
```

```
    2. }
```



1 2 3 2

```

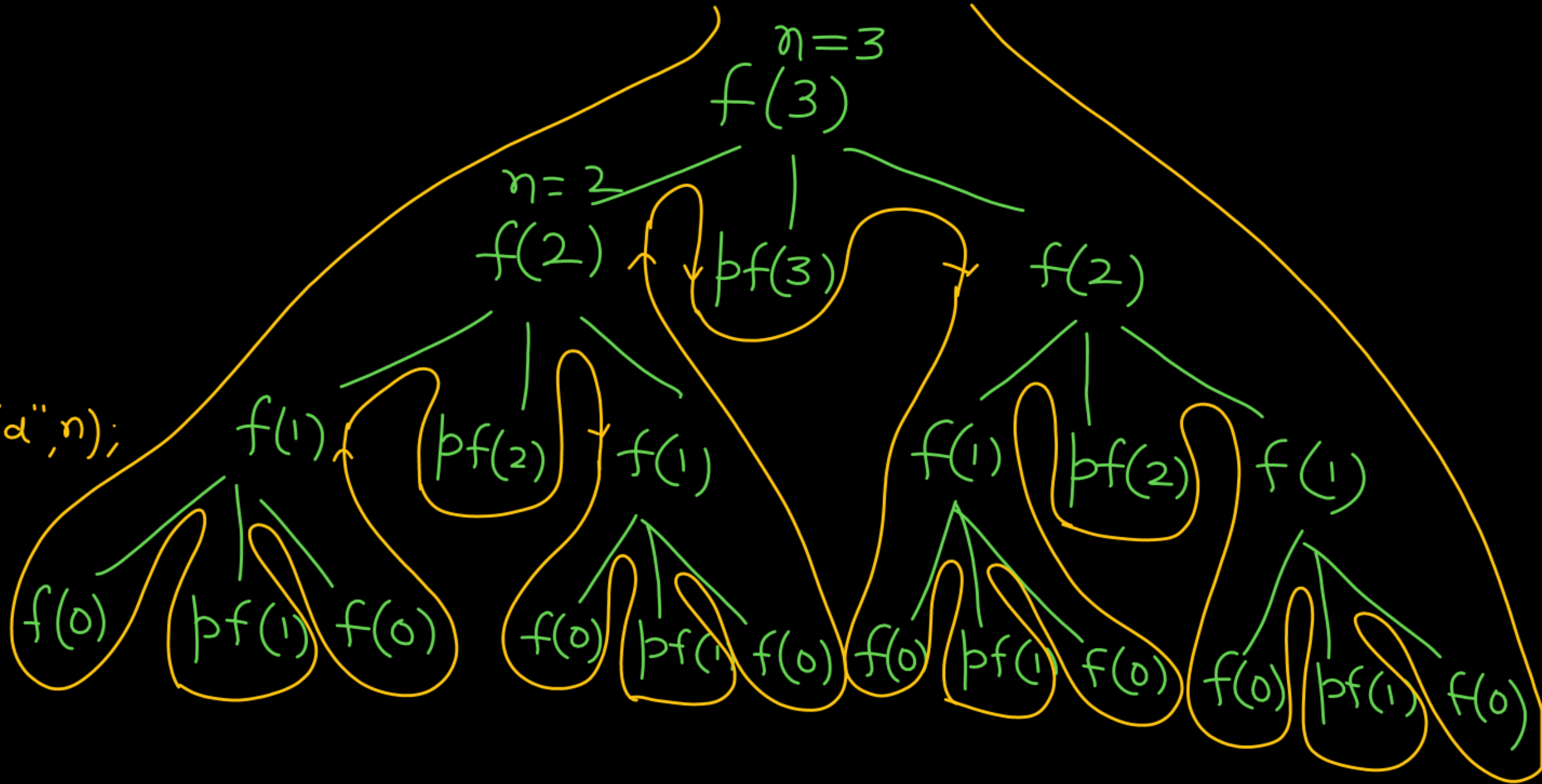
void f(int n)
{
    if(n>0){
        1. f(n-1);
        2. printf("/d",n);
        3. f(n-1);
    }
}

```

```

void main(){
    f(3);
}

```



1 2 1 3 1 2 1

```

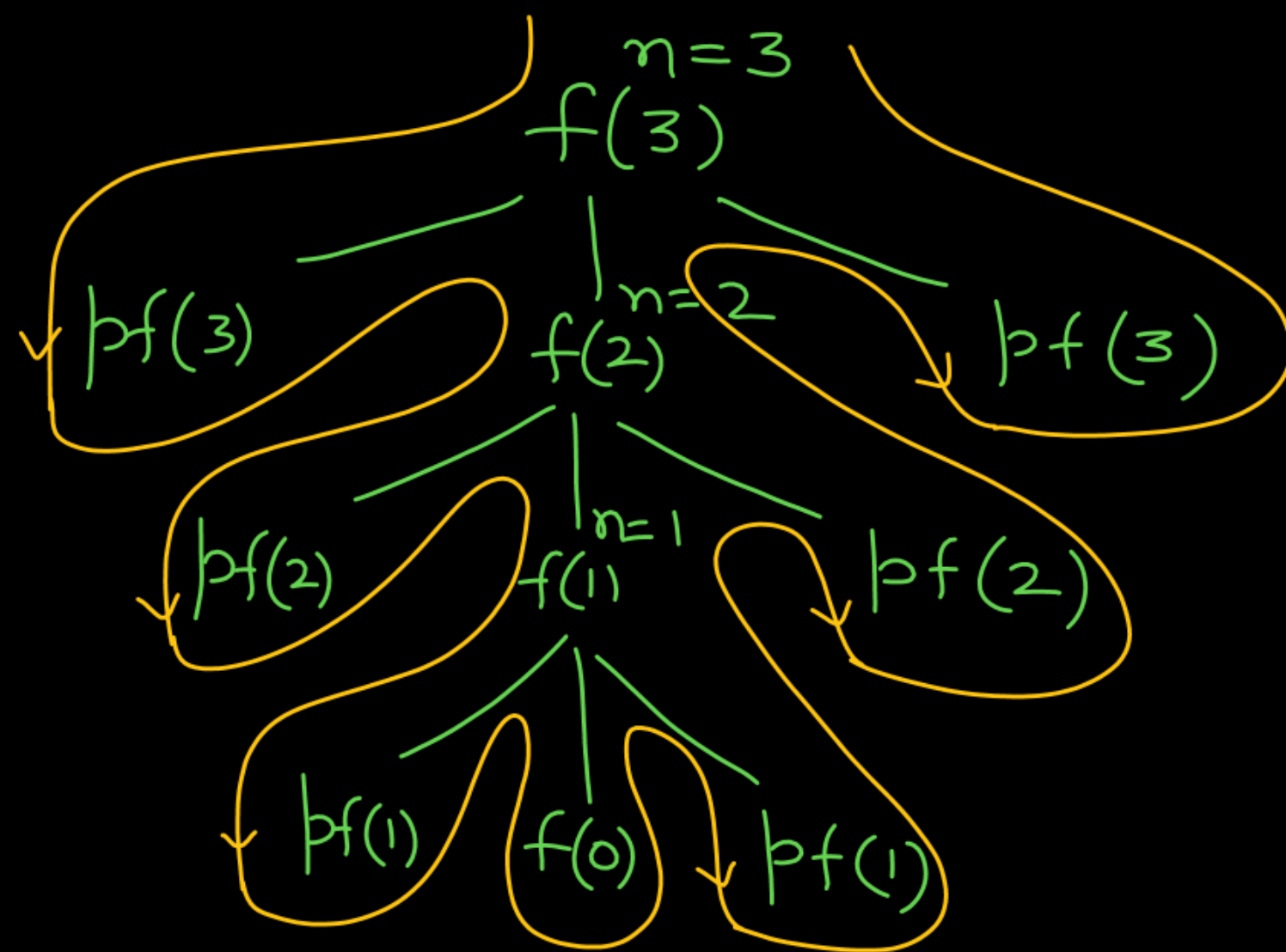
void f(int n){
    if(n>0){
        1. printf("/d",n);
        2. f(n-1);
        3. printf("/d",n);
    }
}

```

```

void main(){
    f(3);
}

```



3 2 1 1 2 3

17 → binary

1)
$$\begin{array}{r} 2 \overline{) 17} \\ \underline{16} \\ 1 \end{array}$$
 8

2)
$$\begin{array}{r} 2 \overline{) 8} \\ \underline{8} \\ 0 \end{array}$$
 4 ✓

3)
$$\begin{array}{r} 2 \overline{) 4} \\ \underline{4} \\ 0 \end{array}$$
 2

2	17	Rem
2	8	1
2	4	0
2	2	0

17 → $n/2$
Rem $n/2$


```

void bin(int n)
{
    if (n is small)  $\Rightarrow n=0 \parallel n=1$ 
    {
        printf("/d", n);
        return;
    }
    else {
        bin(n/2);
        printf("/d", n/2);
    }
}

```

Q	17	Rem
2	8	1
2	4	0
	2	0

Decimal to octal
to hexadecimal
 $a+b$
 $a \times b$

using
recursion

Strong

4 PM

Doubts ? {
* Tower of Hanoi
* Fibonacci Series

```
void bin(int n){
```

```
    if (n == 1 || n == 0)
```

```
    {
```

```
        printf("%d", n);
```

```
        return;
```

```
    }
```

```
else {
```

```
    }
```

```
}
```

17

NTR
% /

1234

1234./10

1234/10

④

①23

12349

12349./10

12349/10

⑨

①234

